

CISD 43 NLP Sentiment Analysis (Lab/Project 1)

```
# Import necessary libraries
# CISD 43 (Spring 2026) at Mt. SAC
# NLP Sentiment Analysis (Lab)
# Assignment worked on by Juan Nevares in Google Colab
# 5/3/2026

# Class references:
# youtube_NBC.ipynb

# Note that print PDF may cut off due to an issue with rendering on Colab:
# https://github.com/googlecolab/colabtools/issues/4357
# https://github.com/googlecolab/colabtools/issues/5505

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import mpl_toolkits
%matplotlib inline
```

1. Install Required Libraries for Text Extraction, Sentiment Analysis, Data Organizing, and Visualization

```
# Install required libraries
!pip install youtube-transcript-api textblob wordcloud matplotlib pandas
```

```
Collecting youtube-transcript-api
  Downloading youtube_transcript_api-1.2.4-py3-none-any.whl.metadata (24 kB)
Requirement already satisfied: textblob in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: wordcloud in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: defusedxml<0.8.0,>=0.7.1 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: nltk>=3.9 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: numpy>=1.19.3 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: pillow in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: cyclers>=0.10 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages
```

```
Requirement already satisfied: click in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: joblib in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: regex>=2021.8.3 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packag
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-pa
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/d
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/d
Downloading youtube_transcript_api-1.2.4-py3-none-any.whl (485 kB)
485.2/485.2 kB 7.4 MB/s eta 0:00:00
Installing collected packages: youtube-transcript-api
Successfully installed youtube-transcript-api-1.2.4
```

2. Begin Import of the Libraries Required for running this project

```
from youtube_transcript_api import YouTubeTranscriptApi
from textblob import TextBlob
from wordcloud import WordCloud, STOPWORDS

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
%matplotlib inline
import re
```

3. Paste YouTube Video Link for parsing and grabbing the video ID of the URL

```
# Note:

# I made a modification to allow a bit more flexibility to try and grab the
# Video Id of any YouTube video, instead of having to parse it manually.

youtube_url = "https://www.youtube.com/watch?v=Fyet0-L1IWI"
split_pattern = r'\?v='
split_result = re.split(split_pattern, youtube_url)
video_id = split_result[1]
video_id

'Fyet0-L1IWI'
```

4. Fetch YouTube Transcript

```
def transcript_entry_to_dict(entry):
    """Convert transcript entry to a dictionary.

    This helps the notebook work with different versions of youtube-transcript-
```

```
"""
if isinstance(entry, dict):
    return {
        "text": entry.get("text", ""),
        "start": entry.get("start", None),
        "duration": entry.get("duration", None)
    }

return {
    "text": getattr(entry, "text", ""),
    "start": getattr(entry, "start", None),
    "duration": getattr(entry, "duration", None)
}

try:
    ytt_api = YouTubeTranscriptApi()
    transcript = ytt_api.fetch(video_id)
    transcript_data = [transcript_entry_to_dict(entry) for entry in transcript]

    print("Transcript successfully loaded.")
    print("Number of transcript entries:", len(transcript_data))

except Exception as e:
    transcript_data = []
    print("The transcript could not be loaded.")
    print("Possible reasons:")
    print("- The video does not have captions or a transcript.")
    print("- The transcript is disabled.")
    print("- The video ID is incorrect.")
    print("- There is a temporary connection or access issue.")
    print()
    print("Error message:")
    print(e)
```

Transcript successfully loaded.
Number of transcript entries: 168

5. Create Dataframe to hold data to read, clean, and analyze

```
df = pd.DataFrame(transcript_data)

if df.empty:
    print("No transcript data is available. Please try another video ID.")
else:
    display(df.head())
```

	text	start	duration
0	Hello everyone and welcome to the workshop! To...	0.80	5.96
1	SBCs available in the market right now to \nse...	6.76	5.08
2	projects. I'll be taking a look at the Raspber...	11.84	7.68
3	Pi Zero 2W. I'll try to keep them all straight...	19.52	6.48
4	it all a little bit more fun, I'll be grading ...	26.00	3.96

6. Clean Transcript Text

```
def clean_text(text):
    """Clean transcript text by removing extra spaces and line breaks."""
    text = str(text)
    text = text.replace("\n", " ")
    text = re.sub(r"\s+", " ", text)
    return text.strip()
```

```
if not df.empty:
    df["clean_text"] = df["text"].apply(clean_text)
    display(df[["text", "clean_text"]].head())
```

	text	clean_text
0	Hello everyone and welcome to the workshop! To...	Hello everyone and welcome to the workshop! To...
1	SBCs available in the market right now to \nse...	SBCs available in the market right now to see ...
2	projects. I'll be taking a look at the Raspber...	projects. I'll be taking a look at the Raspber...
3	Pi Zero 2W. I'll try to keep them all straight...	Pi Zero 2W. I'll try to keep them all straight...

7. Combine Transcript into One Text

```
if not df.empty:
    full_text = " ".join(df["clean_text"].dropna().tolist())

    print("Total characters in transcript:", len(full_text))
    print()
    print(full_text[:500])
else:
    full_text = ""
    print("No transcript text is available.")
```

Total characters in transcript: 16472

Hello everyone and welcome to the workshop! Today I'm going to compare the three

8. Applying Sentiment Analysis

```
def get_polarity(text):
    return TextBlob(text).sentiment.polarity

def get_subjectivity(text):
    return TextBlob(text).sentiment.subjectivity

if not df.empty:
    df["polarity"] = df["clean_text"].apply(get_polarity)
    df["subjectivity"] = df["clean_text"].apply(get_subjectivity)

    display(df[["clean_text", "polarity", "subjectivity"]].head())
```

	clean_text	polarity	subjectivity
0	Hello everyone and welcome to the workshop! To...	0.305556	0.544444
1	SBCs available in the market right now to see ...	0.228571	0.353571
2	projects. I'll be taking a look at the Raspber...	0.000000	0.000000
3	Pi Zero 2W. I'll try to keep them all straight...	0.200000	0.400000
4	it all a little bit more fun, I'll be grading ...	0.153125	0.450000

9. Interpret Sentiment

```
def sentiment_label(score):
    if score > 0:
        return "Positive"
    elif score < 0:
        return "Negative"
    else:
        return "Neutral"

if not df.empty:
    df["sentiment"] = df["polarity"].apply(sentiment_label)
    display(df[["clean_text", "polarity", "sentiment"]].head())
```

	clean_text	polarity	sentiment
0	Hello everyone and welcome to the workshop! To...	0.305556	Positive
1	SBCs available in the market right now to see ...	0.228571	Positive
2	projects. I'll be taking a look at the Raspber...	0.000000	Neutral
3	Pi Zero 2W. I'll try to keep them all straight...	0.200000	Positive
4	it all a little bit more fun, I'll be grading ...	0.153125	Positive

10. Sentiment Summary

```
if not df.empty:
    sentiment_counts = df["sentiment"].value_counts().reset_index()
    sentiment_counts.columns = ["Sentiment", "Count"]
    display(sentiment_counts)
```

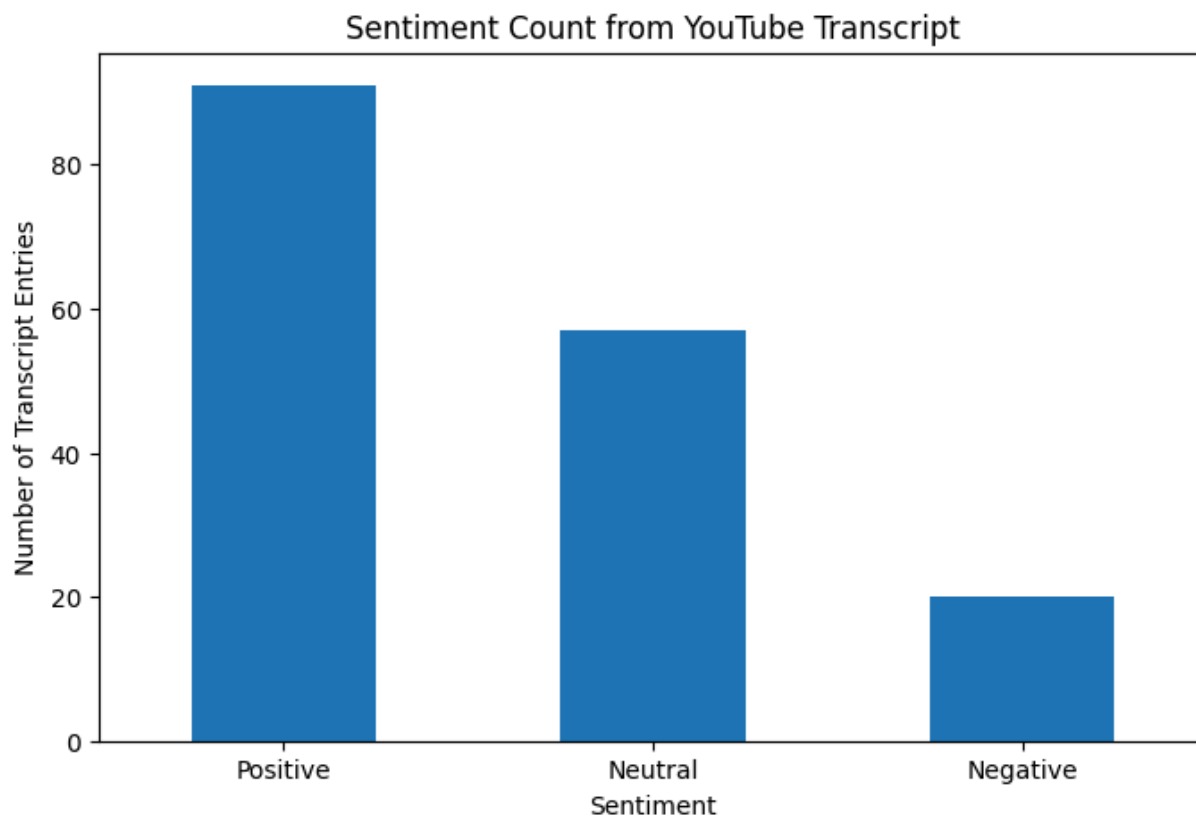
	Sentiment	Count
0	Positive	91
1	Neutral	57
2	Negative	20

```
df_stats = df.groupby("sentiment")["polarity"].describe()
df_stats
```

	count	mean	std	min	25%	50%	75%	n
sentiment								
Negative	20.0	-0.216530	0.166102	-0.625000	-0.303125	-0.191667	-0.093229	-0
Neutral	57.0	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0
Positive	91.0	0.339121	0.220243	0.015278	0.181250	0.300000	0.487500	1

11. Visualize Sentiment Counts

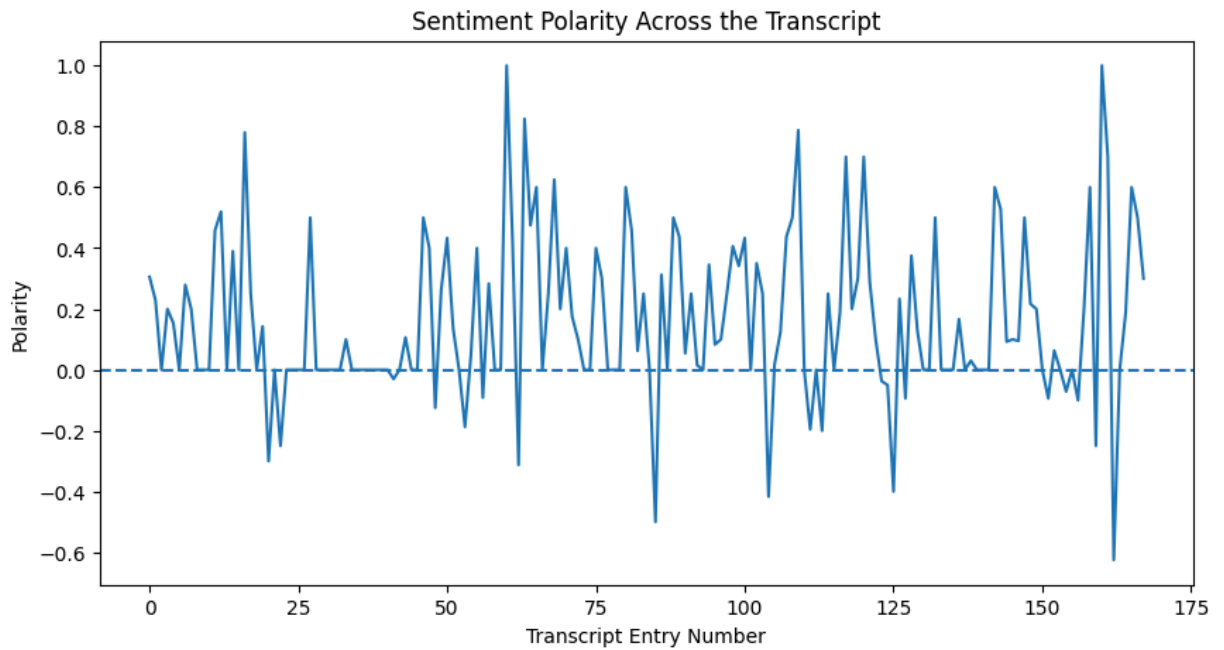
```
if not df.empty:
    sentiment_counts.plot(kind="bar", x="Sentiment", y = "Count", legend=False,
    plt.title("Sentiment Count from YouTube Transcript")
    plt.xlabel("Sentiment")
    plt.ylabel("Number of Transcript Entries")
    plt.xticks(rotation=0)
    plt.show()
```



12. Visualize the Polarity of transcript Over Time

- List item
- List item

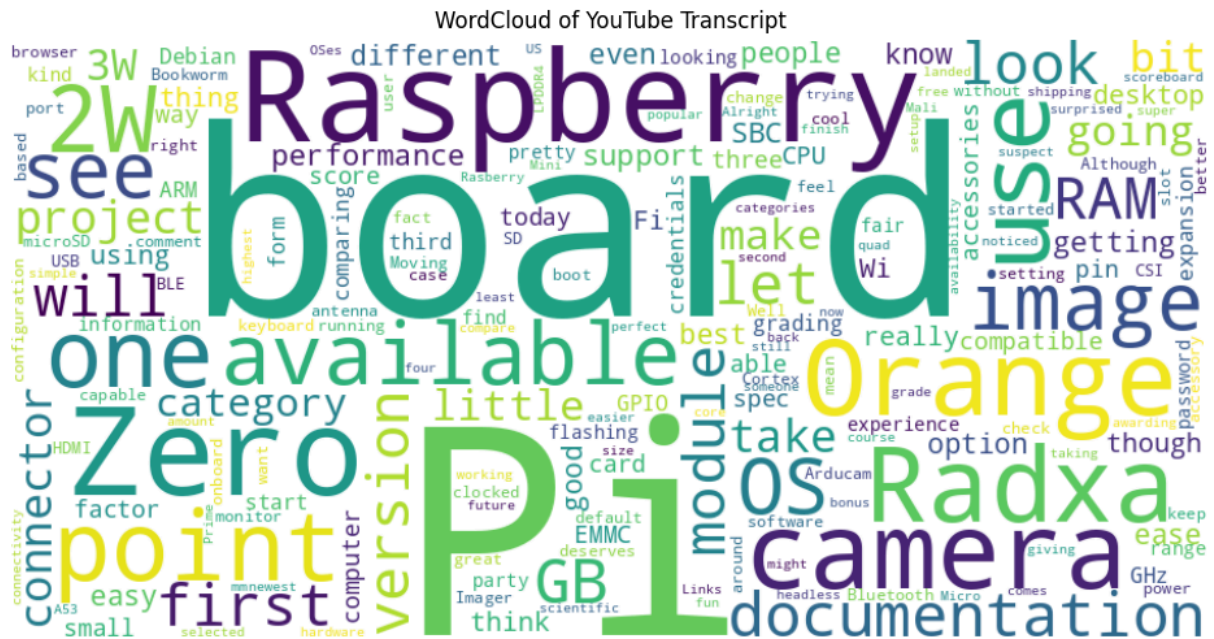
```
if not df.empty:
    plt.figure(figsize=(10, 5))
    plt.plot(df.index, df["polarity"])
    plt.axhline(0, linestyle="--")
    plt.title("Sentiment Polarity Across the Transcript")
    plt.xlabel("Transcript Entry Number")
    plt.ylabel("Polarity")
    plt.show()
```



```
if full_text.strip():
    custom_stopwords = set(STOPWORDS)

    wordcloud = WordCloud(
        width=900,
        height=450,
        background_color="white",
        stopwords=custom_stopwords,
        collocations=False
    ).generate(full_text)

    plt.figure(figsize=(12,6))
    plt.imshow(wordcloud, interpolation="bilinear")
    plt.axis("off")
    plt.title("WordCloud of YouTube Transcript")
    plt.show()
else:
    print("No transcript text is available for the WordCloud.")
```



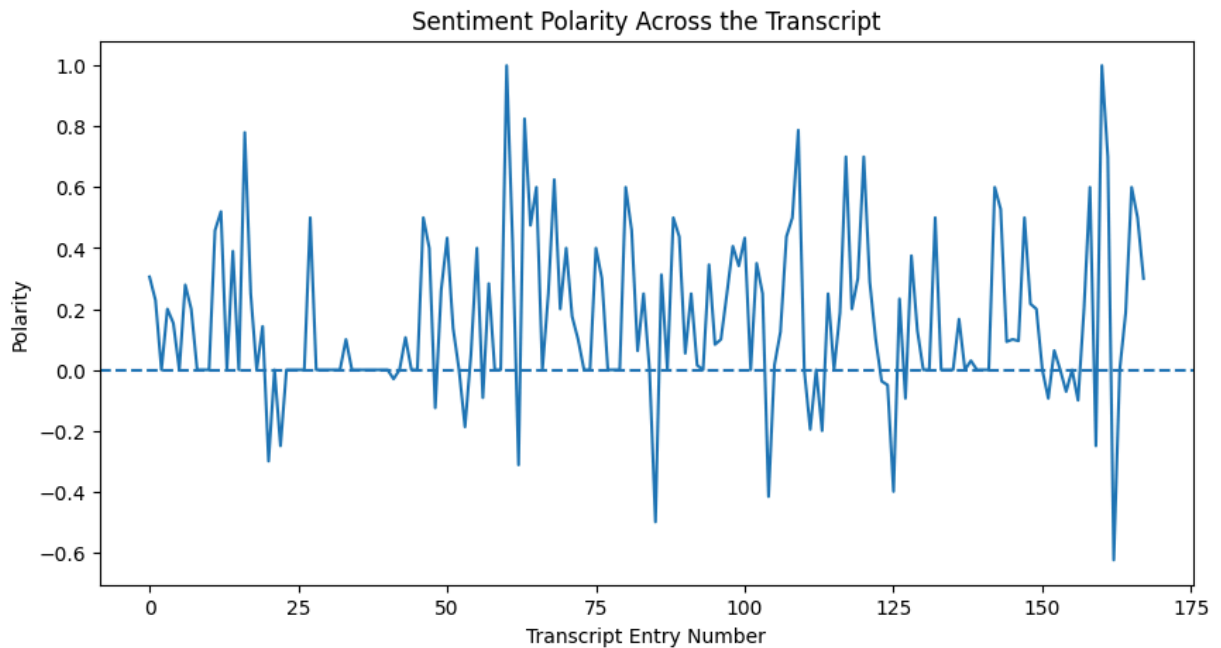
- Findings and Conclusions:

I intentionally looked into the selection of a YouTube Review on different electronic products because I knew that it would offer a lot of subjectivity and polarity towards their evaluations. Whenever thinking about comparisons, we often think about what's good or bad about the idea(s) or object(s) that are being looked into themselves, or with others.

However, in the context of having viewers make their own assumptions and not be too biased in their conclusions from watching, it's likely that the person(s) credited in the video review(s) would not want to lean towards too good or too bad, and likely would try to keep the evaluations/review findings as balanced as possible.

When looking at the chart again below, from the initial analysis, we can how varied it goes back in forth between Negative to Positive, likely a sign that the content creator(s) evaluating/reporting their opinions on the findings are attempting to keep it as balanced as possible.

```
if not df.empty:
    plt.figure(figsize=(10, 5))
    plt.plot(df.index, df["polarity"])
    plt.axhline(0, linestyle="--")
    plt.title("Sentiment Polarity Across the Transcript")
    plt.xlabel("Transcript Entry Number")
    plt.ylabel("Polarity")
    plt.show()
```



All this being said, there may be exceptions to this rule if it's (object or idea) is a more objectively good or bad in many other reviews or contexts within the community or world.

One thing I would like to look into (which, just occurred to me while writing this), in the future is maybe:

1. Keywords referencing each respective product.
2. Grab sentences with each respective product mentioned.
3. Then, conduct analysis' into each one with the similar steps performed above.

This may provide a more narrowed down review of each product, based on their own NLP and Sentiment Analysis'.

